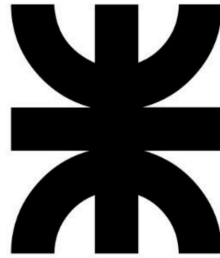


Universidad Tecnológica Nacional
Facultad Regional Concepción del Uruguay



Bases de Datos

Grupo: Patos

Integrantes:

Bogado María Florencia
Canela Miranda
Garnier Anna
Gomez Serena
Villaverde Lara

Docentes:

Ing. Francisco Fazzio
Ing. Pablo Colombo

Fecha de entrega: 23/06/2026

ÍNDICE

INTRODUCCIÓN.....	3
MODELO ENTIDAD-RELACIÓN (DER).....	4
MODELO RELACIONAL.....	4
Descripción de atributos, tipos y constraints por tabla.....	5
Índices de optimización.....	8
Script SQL — CREATE TABLE.....	9
DECISIONES DE DISEÑO.....	13
DESCRIPCIÓN STORED PROCEDURES.....	14
DESCRIPCIÓN TRIGGERS.....	16
ARQUITECTURA DEL AGENTE IA.....	17
PROMPT DEL SISTEMA UTILIZADO.....	20
EVALUACIÓN DEL AGENTE.....	22
MÓDULO DE RECOMENDACIONES.....	24
CONCLUSIONES.....	26
BIBLIOGRAFÍA.....	28

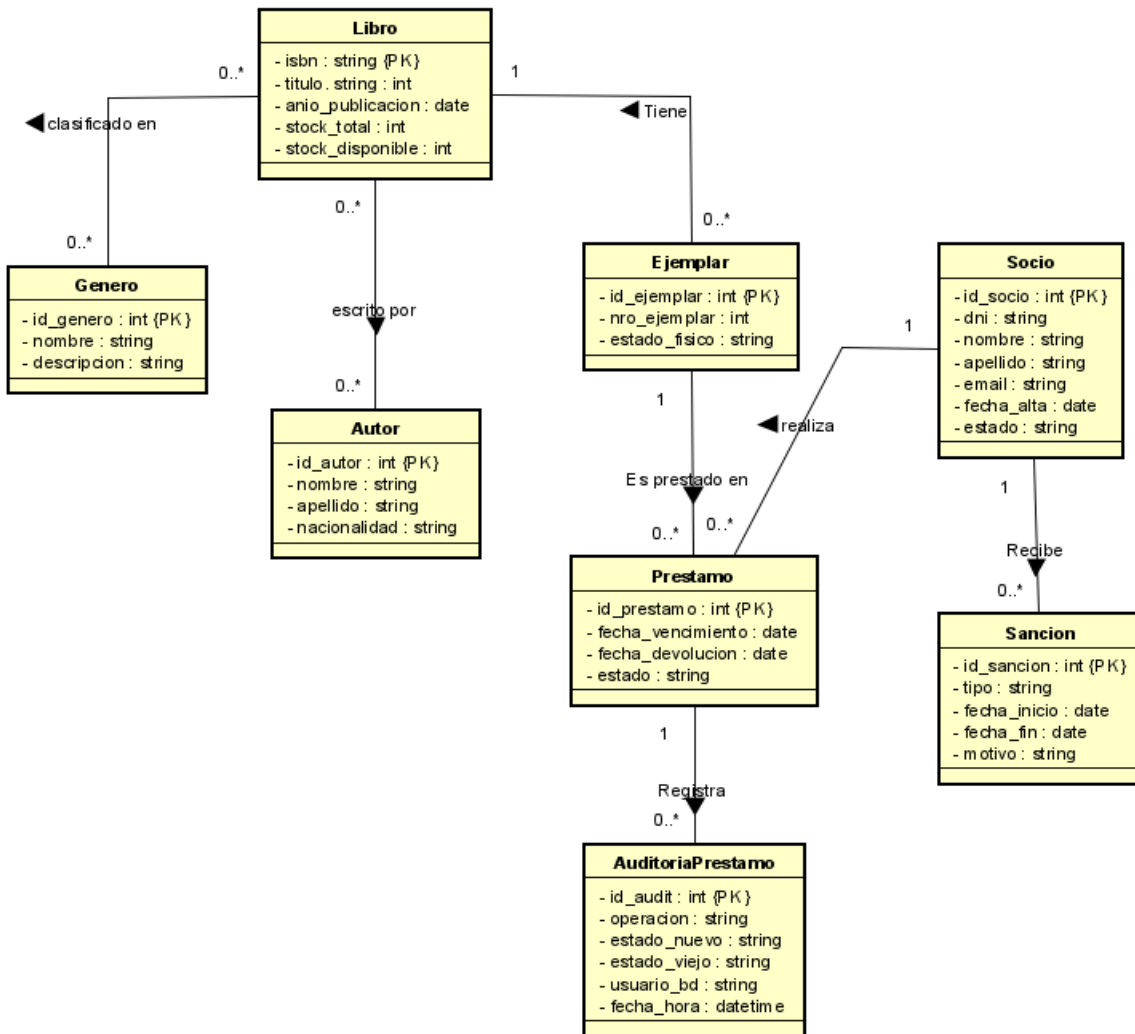
INTRODUCCIÓN

Antes de que existiera la inteligencia artificial, si se quería implementar un sistema que pudiera responder preguntas sobre sus datos, había que programar cada consulta a mano. Si alguien quería saber qué libros estaban disponibles, un programador tenía que haber escrito esa consulta específica con anticipación. Cualquier pregunta que no estuviera programada, no se podía responder.

BiblioIA es un sistema de gestión de biblioteca que combina una base de datos relacional MySQL con un agente de inteligencia artificial capaz de interpretar preguntas en lenguaje natural (español), traducirlas a consultas SQL y generar recomendaciones personalizadas de libros para los socios.

El sistema cuenta con una base de datos que registra socios, libros, ejemplares, préstamos y sanciones, y un agente de inteligencia artificial que permite hacerle preguntas en español y obtener respuestas en base a esos datos.

El objetivo principal del proyecto es que ambas partes funcionen integradas: la base de datos con toda su lógica de negocio, y el agente capaz de interpretarla. Además, se implementan stored procedures y triggers para automatizar procesos como la generación de sanciones o la actualización del stock, y desarrollar un módulo de recomendaciones personalizadas según el historial de cada socio.

MODELO ENTIDAD-RELACIÓN (DER)**MODELO RELACIONAL**

A continuación se presenta el esquema en notación relacional estándar. Las claves primarias se indican con PK, las foráneas con FK, y los atributos con restricción de unicidad con UQ.

Tabla	Esquema relacional
GENERO	(id_genero PK, nombre UQ NOT NULL, descripcion)
AUTOR	(id_autor PK, nombre NOT NULL, apellido NOT NULL, nacionalidad)
LIBRO	(isbn PK, titulo NOT NULL, anio_publicacion NOT NULL, stock_total, stock_disponible)
LIBRO_AUTOR	(isbn PK FK→LIBRO, id_autor PK FK→AUTOR) — N:M
LIBRO_GENERO	(isbn PK FK→LIBRO, id_genero PK FK→GENERO) — N:M

Tabla	Esquema relacional
SOCIO	(id_socio PK, dni UQ NOT NULL, nombre NOT NULL, apellido NOT NULL, email UQ NOT NULL, fecha_alta, estado)
EJEMPLAR	(id_ejemplar PK, isbn FK→LIBRO, nro_ejemplar, estado_físico)
PRESTAMO	(id_prestamo PK, id_socio FK→SOCIO, id_ejemplar FK→EJEMPLAR, fecha_prestamo, fecha_vencimiento, fecha_devolucion, estado)
SANCION	(id_sancion PK, id_socio FK→SOCIO, tipo, fecha_inicio, fecha_fin, motivo)
AUDITORIA_PRESTAMOS	(id_audit PK, id_prestamo, operacion, estado_nuevo, estado_viejo, usuario_bd, fecha_hora)

Descripción de atributos, tipos y constraints por tabla

Se detalla cada relación del modelo con sus atributos, tipos de datos y restricciones de integridad.

GENERO

Catálogo de géneros literarios disponibles en la biblioteca.

Atributo	Tipo	Constraints	Notas
id_genero	INT	PK · NOT NULL · AUTO_INCREMENT	Identificador único del género
nombre	VARCHAR(60)	NOT NULL · UNIQUE	Nombre sin repetición
descripcion	VARCHAR(255)	—	Campo opcional

AUTOR

Registro de autores de los libros del catálogo.

Atributo	Tipo	Constraints	Notas
id_autor	INT	PK · NOT NULL · AUTO_INCREMENT	Identificador único del autor
nombre	VARCHAR(80)	NOT NULL	—
apellido	VARCHAR(80)	NOT NULL	—
nacionalidad	VARCHAR(60)	—	Campo opcional

LIBRO

Catálogo de títulos disponibles.

Atributo	Tipo	Constraints	Notas
isbn	VARCHAR(20)	PK · NOT NULL	Identificador estándar ISBN
título	VARCHAR(200)	NOT NULL	Índice: idx_libro_titulo

Atributo	Tipo	Constraints	Notas
anio_publicacion	YEAR	NOT NULL · CHECK (1000–2100)	Rango válido de años
stock_total	SMALLINT	NOT NULL · CHECK (≥ 0)	Default 1
stock_disponible	SMALLINT	NOT NULL · CHECK (≥ 0 y $\leq$ stock_total)	Default 1

LIBRO_AUTOR (N:M)

Tabla intermedia que resuelve la relación muchos a muchos entre libros y autores.

Atributo	Tipo	Constraints	Notas
isbn	VARCHAR(20)	PK · FK → LIBRO	ON DELETE CASCADE · ON UPDATE CASCADE
id_autor	INT	PK · FK → AUTOR	ON DELETE CASCADE · ON UPDATE CASCADE

LIBRO_GENERO (N:M)

Tabla intermedia que resuelve la relación muchos a muchos entre libros y géneros.

Atributo	Tipo	Constraints	Notas
isbn	VARCHAR(20)	PK · FK → LIBRO	ON DELETE CASCADE · ON UPDATE CASCADE
id_genero	INT	PK · FK → GENERO	ON DELETE CASCADE · ON UPDATE CASCADE

SOCIO

Miembros activos o inactivos de la biblioteca que pueden solicitar préstamos.

Atributo	Tipo	Constraints	Notas
id_socio	INT	PK · NOT NULL · AUTO_INCREMENT	Identificador interno
dni	VARCHAR(15)	NOT NULL · UNIQUE	Índice: idx_socio_dni
nombre	VARCHAR(80)	NOT NULL	—
apellido	VARCHAR(80)	NOT NULL	—
email	VARCHAR(120)	NOT NULL · UNIQUE	Índice: idx_socio_email
fecha_alta	DATE	NOT NULL	Default: CURRENT_DATE

Atributo	Tipo	Constraints	Notas
estado	VARCHAR(12)	NOT NULL · CHECK	ACTIVO / SUSPENDIDO / BAJA

EJEMPLAR

Copia física individual de un libro. Un mismo ISBN puede tener múltiples ejemplares.

Atributo	Tipo	Constraints	Notas
id_ejemplar	INT	PK · NOT NULL · AUTO_INCREMENT	Identificador único de ejemplar
isbn	VARCHAR(20)	NOT NULL · FK → LIBRO	ON DELETE RESTRICT · Índice: idx_ejemplar_isbn
nro_ejemplar	SMALLINT	NOT NULL · UNIQUE (isbn, nro)	Número único por ISBN
estado_fisico	VARCHAR(12)	NOT NULL · CHECK	BUENO / DETERIORADO / BAJA

PRESTAMO

Registro de cada préstamo de un ejemplar a un socio, con control de fechas y estado.

Atributo	Tipo	Constraints	Notas
id_prestamo	INT	PK · NOT NULL · AUTO_INCREMENT	Identificador único del préstamo
id_socio	INT	NOT NULL · FK → SOCIO	ON DELETE RESTRICT · Índice: idx_prestamo_socio
id_ejemplar	INT	NOT NULL · FK → EJEMPLAR	ON DELETE RESTRICT · Índice: idx_prestamo_ejemplar
fecha_prestamo	DATE	NOT NULL	Default: CURRENT_DATE
fecha_vencimiento	DATE	NOT NULL · CHECK	> fecha_prestamo
fecha_devolucion	DATE	CHECK	NULL hasta devolver; >= fecha_prestamo
estado	VARCHAR(12)	NOT NULL · CHECK	ACTIVO / DEVUELTO / VENCIDO · Índice: idx_prestamo_estado

SANCION

Penalidades aplicadas a un socio por mora, daño o pérdida de ejemplares.

Atributo	Tipo	Constraints	Notas
id_sancion	INT	PK · NOT NULL · AUTO_INCREMENT	Identificador único
id_socio	INT	NOT NULL · FK → SOCIO	ON DELETE CASCADE, · Índice: idx_sancion_socio
tipo	VARCHAR(20)	NOT NULL · CHECK	MORA / DAÑO / PERDIDA / OTRO
fecha_inicio	DATE	NOT NULL	Default: CURRENT_DATE
fecha_fin	DATE	NOT NULL · CHECK	>= fecha_inicio
motivo	VARCHAR(255)	—	Campo opcional

AUDITORIA_PRESTAMOS

Log automático de cambios sobre la tabla PRÉSTAMO. Registra INSERT, UPDATE y DELETE.

Atributo	Tipo	Constraints	Notas
id_audit	INT	PK · NOT NULL · AUTO_INCREMENT	Identificador del registro
id_prestamo	INT	NOT NULL	Referencia histórica (sin FK para conservar datos)
operacion	VARCHAR(10)	NOT NULL · CHECK	INSERT / UPDATE / DELETE
estado_nuevo	VARCHAR(12)	—	Nuevo estado tras la operación
estado_viejo	VARCHAR(12)	—	Estado anterior a la operación
usuario_bd	VARCHAR(80)	NOT NULL	Default: CURRENT_USER()
fecha_hora	DATETIME	NOT NULL	Default: CURRENT_TIMESTAMP

Índices de optimización

Además de las claves primarias (que generan índice automáticamente), se definieron los siguientes índices secundarios para optimizar las búsquedas más frecuentes:

Índice	Tabla	Columna(s)	Propósito
idx_libro_titulo	LIBRO	titulo	Acelera la búsqueda de libros por título
idx_socio_dni	SOCIO	dni	Acelera la búsqueda de socios por DNI
idx_socio_email	SOCIO	email	Acelera la búsqueda de socios por email
idx_ejemplar_isbn	EJEMPLAR	isbn	Acelera la búsqueda de ejemplares de un libro
idx_prestamo_socio	PRESTAMO	id_socio	Acelera la consulta de préstamos por socio
idx_prestamo_ejemplar	PRESTAMO	id_ejemplar	Acelera la consulta de préstamos por ejemplar
idx_prestamo_estado	PRESTAMO	estado	Acelera el filtrado por estado (ACTIVO/DEVUELTO/VENCIDO)
idx_sancion_socio	SANCION	id_socio	Acelera la consulta de sanciones por socio

Script SQL — CREATE TABLE

El siguiente script fue ejecutado en MySQL para crear la estructura completa de la base de datos. Incluye definición de tipos, restricciones de integridad, claves primarias y foráneas, e índices de optimización.

```
CREATE TABLE GENERO (
id_genero INT NOT NULL AUTO_INCREMENT,
nombre VARCHAR(60) NOT NULL,
descripcion VARCHAR(255),
CONSTRAINT pk_genero PRIMARY KEY (id_genero),
CONSTRAINT uq_genero_nom UNIQUE (nombre)
);

CREATE TABLE AUTOR (
id_autor INT NOT NULL AUTO_INCREMENT,
nombre VARCHAR(80) NOT NULL,
apellido VARCHAR(80) NOT NULL,
nacionalidad VARCHAR(60),
CONSTRAINT pk_autor PRIMARY KEY (id_autor)
```

```
);

CREATE TABLE LIBRO (
    isbn VARCHAR(20) NOT NULL,
    titulo VARCHAR(200) NOT NULL,
    anio_publicacion SMALLINT NOT NULL,
    stock_total SMALLINT NOT NULL DEFAULT 1,
    stock_disponible SMALLINT NOT NULL DEFAULT 1,
    CONSTRAINT pk_libro PRIMARY KEY (isbn),
    CONSTRAINT chk_stock_total_pos CHECK (stock_total >= 0),
    CONSTRAINT chk_stock_disp_pos CHECK (stock_disponible >= 0),
    CONSTRAINT chk_stock_disp_lte_tot CHECK (stock_disponible <=
stock_total),
    CONSTRAINT chk_anio_pub CHECK (anio_publicacion BETWEEN 1000 AND 2100)
);

CREATE INDEX idx_libro_titulo ON LIBRO (titulo);

CREATE TABLE LIBRO_AUTOR (
    isbn VARCHAR(20) NOT NULL,
    id_autor INT NOT NULL,
    CONSTRAINT pk_libro_autor PRIMARY KEY (isbn, id_autor),
    CONSTRAINT fk_la_isbn FOREIGN KEY (isbn) REFERENCES LIBRO (isbn) ON
DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_la_autor FOREIGN KEY (id_autor) REFERENCES AUTOR
(id_autor) ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE LIBRO_GENERO (
    isbn VARCHAR(20) NOT NULL,
    id_genero INT NOT NULL,
    CONSTRAINT pk_libro_genero PRIMARY KEY (isbn, id_genero),
    CONSTRAINT fk_lg_isbn
```

```
FOREIGN KEY (isbn) REFERENCES LIBRO (isbn) ON DELETE CASCADE ON UPDATE
CASCADE,

CONSTRAINT fk_lg_genero FOREIGN KEY (id_genero) REFERENCES GENERO
(id_genero) ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE SOCIO (
id_socio INT NOT NULL AUTO_INCREMENT,
dni VARCHAR(15) NOT NULL,
nombre VARCHAR(80) NOT NULL,
apellido VARCHAR(80) NOT NULL,
email VARCHAR(120) NOT NULL,
fecha_alta DATE NOT NULL DEFAULT (CURRENT_DATE),
estado VARCHAR(12) NOT NULL DEFAULT 'ACTIVO',
CONSTRAINT pk_socio PRIMARY KEY (id_socio),
CONSTRAINT uq_socio_dni UNIQUE (dni),
CONSTRAINT uq_socio_email UNIQUE (email),
CONSTRAINT chk_socio_estado CHECK (estado IN ('ACTIVO', 'SUSPENDIDO',
'BAJA'))
);

CREATE INDEX idx_socio_dni ON SOCIO (dni);
CREATE INDEX idx_socio_email ON SOCIO (email);

CREATE TABLE EJEMPLAR (
id_ejemplar INT NOT NULL AUTO_INCREMENT,
isbn VARCHAR(20) NOT NULL,
nro_ejemplar SMALLINT NOT NULL,
estado_fisico VARCHAR(12) NOT NULL DEFAULT 'BUENO',
CONSTRAINT pk_ejemplar PRIMARY KEY (id_ejemplar),
CONSTRAINT uq_ejemplar_isbn_nro UNIQUE (isbn, nro_ejemplar),
CONSTRAINT fk_ejemplar_isbn FOREIGN KEY (isbn) REFERENCES LIBRO (isbn)
ON DELETE RESTRICT ON UPDATE CASCADE,
CONSTRAINT chk_estado_fisico CHECK (estado_fisico IN ('BUENO',
'DETERIORADO', 'BAJA'))
);
```

```
CREATE INDEX idx_ejemplar_isbn ON EJEMPLAR (isbn);

CREATE TABLE PRESTAMO (
  id_prestamo INT NOT NULL AUTO_INCREMENT,
  id_socio INT NOT NULL,
  id_ejemplar INT NOT NULL,
  fecha_prestamo DATE NOT NULL DEFAULT (CURRENT_DATE),
  fecha_vencimiento DATE NOT NULL,
  fecha_devolucion DATE,
  estado VARCHAR(12) NOT NULL DEFAULT 'ACTIVO',
  CONSTRAINT pk_prestamo PRIMARY KEY (id_prestamo),
  CONSTRAINT fk_prest_socio
  FOREIGN KEY (id_socio) REFERENCES SOCIO (id_socio) ON DELETE RESTRICT
  ON UPDATE CASCADE,
  CONSTRAINT fk_prest_ejemplar
  FOREIGN KEY (id_ejemplar) REFERENCES EJEMPLAR (id_ejemplar) ON DELETE
  RESTRICT ON UPDATE CASCADE,
  CONSTRAINT chk_prestamo_estado CHECK (estado IN ('ACTIVO', 'DEVUELTO',
  'VENCIDO')),
  CONSTRAINT chk_venc_after_prest CHECK (fecha_vencimiento >
  fecha_prestamo),
  CONSTRAINT chk_devol_after_prest CHECK (fecha_devolucion IS NULL OR
  fecha_devolucion >= fecha_prestamo) );

CREATE INDEX idx_prestamo_socio ON PRESTAMO (id_socio);
CREATE INDEX idx_prestamo_ejemplar ON PRESTAMO (id_ejemplar);
CREATE INDEX idx_prestamo_estado ON PRESTAMO (estado);

CREATE TABLE SANCION (
  id_sancion INT NOT NULL AUTO_INCREMENT,
  id_socio INT NOT NULL,
  tipo VARCHAR(20) NOT NULL DEFAULT 'MORA',
  fecha_inicio DATE NOT NULL DEFAULT (CURRENT_DATE),
```

```

fecha_fin DATE NOT NULL,
motivo VARCHAR(255),
CONSTRAINT pk_sancion PRIMARY KEY (id_sancion),
CONSTRAINT fk_sancion_socio
FOREIGN KEY (id_socio) REFERENCES SOCIO (id_socio) ON DELETE CASCADE ON
UPDATE CASCADE,
CONSTRAINT chk_sancion_tipo CHECK (tipo IN ('MORA', 'DAÑO', 'PERDIDA',
'OTRO')),
CONSTRAINT chk_sancion_fin CHECK (fecha_fin >= fecha_inicio)
);

CREATE INDEX idx_sancion_socio ON SANCION (id_socio);

CREATE TABLE AUDITORIA_PRESTAMOS (
id_audit INT NOT NULL AUTO_INCREMENT,
id_prestamo INT NOT NULL,
operacion VARCHAR(10) NOT NULL,
estado_nuevo VARCHAR(12),
estado_viejo VARCHAR(12),
usuario_bd VARCHAR(80) NOT NULL,
fecha_hora DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT pk_audit PRIMARY KEY (id_audit),
CONSTRAINT chk_audit_op CHECK (operacion IN ('INSERT', 'UPDATE',
'DELETE'))
);

```

DECISIONES DE DISEÑO

Tipos de datos:

Se usó VARCHAR para isbn porque el formato estándar incluye guiones y letras. Los campos de estado se definieron como VARCHAR con CHECK en lugar de ENUM para mayor portabilidad. Se eligió SMALLINT para stocks y año de publicación por ser valores de rango acotado. Las fechas usan DATE salvo en la auditoría donde se usa DATETIME para registrar el momento exacto de cada operación.

Índices:

Se crearon índices en las columnas más consultadas: dni y email en SOCIO, isbn en EJEMPLAR y LIBRO, titulo en LIBRO, y id_socio, id_ejemplar y estado en PRESTAMO. Estos campos son los más usados como filtros tanto por los stored procedures como por el agente Text-to-SQL.

Políticas de claves foráneas:

Se usó RESTRICT en las relaciones donde eliminar el registro padre rompería la trazabilidad histórica (PRESTAMO → SOCIO, PRESTAMO → EJEMPLAR, EJEMPLAR → LIBRO). Se usó CASCADE en las tablas intermedias N:M (LIBRO_AUTOR, LIBRO_GENERO) para limpiar automáticamente las relaciones al eliminar un libro. SANCION → SOCIO usa CASCADE porque las sanciones pierden sentido al dar de baja un socio.

DESCRIPCIÓN STORED PROCEDURES

Para garantizar la integridad de los datos, los procedimientos que se han desarrollado realizan todas las lecturas y validaciones previas antes de invocar START TRANSACTION, minimizando el tiempo de bloqueo de los registros (database locks) y evitando el consumo innecesario de recursos en operaciones destinadas a fallar. También cuentan con bloques DECLARE EXIT HANDLER FOR SQLEXCEPTION para detectar errores y ejecutar ROLLBACK para asegurar la atomicidad del sistema.

Gestión de préstamos y devoluciones**1. sp_registrar_prestamo(IN idsocio INT, IN idejemplar INT)**

Objetivo: registrar un nuevo préstamo en el sistema asignando un plazo estándar de 14 días corridos.

- Verifica que el socio no se encuentre en estado 'SUSPENDIDO'.
- Valida que el socio no supere el límite institucional de tres préstamos activos en simultáneo.
- Constata que el ejemplar físico solicitado no esté dado de 'BAJA'.
- Comprueba que el título del libro posea stock disponible en el inventario.

Si todas las condiciones se cumplen, inicia la transacción, inserta el registro en la tabla PRESTAMO en estado 'ACTIVO' y delega la reducción del stock disponible al trigger trg_actualizar_stock.

2. sp_registrar_devolucion(IN idprestamo INT)

Objetivo: registrar devolución del ejemplar, calcular la existencia de mora y aplicar la sanción si corresponde.

- Verifica que el préstamo actual exista y que su estado actual no sea 'DEVUELTO'.

De esta manera, se evitan duplicados.

Modifica el préstamo a estado 'DEVUELTO' e inserta fecha de transacción, lo cual modifica el stock del libro mediante el trigger trg_actualizar_stock.

Luego, evalúa si la fecha actual superó la fecha de vencimiento original. De ser así, calcula los días de retraso (DATEDIFF) e invoca al procedimiento sp_generar_sanción.

3. sp_generar_sancion(IN idsocio INT, IN tiposancion VARCHAR(20), IN diaspora INT)

Objetivo: automatizar la aplicación de sanciones a los socios en mora.

Suspende al socio durante dos días por cada día de retraso acumulado (diaspora * 2). Inserta el registro en la tabla SANCION y calcula la fecha de finalización de la penalidad a partir del día de la fecha.

4. sp_renovar_prestamo(IN idprestamo INT)

Objetivo: otorgar una extensión de tiempo a un préstamo vigente sin necesidad de realizar una devolución física.

- Verifica que el préstamo esté en estado 'ACTIVO' (no vencido ni devuelto previamente).
- Valida que el socio no registre suspensiones vigentes al momento de la solicitud.

Finalizadas las verificaciones, ejecuta UPDATE sobre la tabla PRESTAMO, extendiendo la fecha de vencimiento original por un periodo de 14 días.

Procesamiento batch y automatización

5. sp_procesar_vencimientos_diarios()

Objetivo: actualiza de forma masiva el estado de los préstamos del sistema para ejecuciones nocturnas o tareas automatizadas de fondo.

Evalúa todos los registros en estado 'ACTIVO' cuya fecha de vencimiento sea inferior a la actual (CURDATE()), cambiando su estado a 'VENCIDO'.

De esta manera, usa la función ROW_COUNT() para devolver un reporte del volumen de registros que se actualizaron durante la corrida del lote.

Seguridad y privacidad

6. sp_anonimizar_socio_baja(IN idsocio INT)

Objetivo: cumplir con las normativas vigentes de protección de datos personales (RGDP / Ley de Protección de Datos Personales 25.326) permitiendo el anonimato de un usuario sin romper la integridad referencial de los registros históricos.

Sobrescribe los campos de identificación (dni, nombre, apellido, email) con cadenas genéricas. Modifica el estado del usuario a 'BAJA' y se preserva la clave primaria para mantener la consistencia de los registros.

Analíticas

7. sp_reporte_salud_biblioteca()

Objetivo: consultar indicadores de rendimiento (KPIs) en tiempo real para la toma de decisiones estratégicas de la administración. Las métricas que se calculan son:

- Total inventario: volumen absoluto de libros físicos registrados.
- Ejemplares en circulación: cantidad de libros en posesión de los usuarios 'ACTIVO' y 'VENCIDO'.
- Tasa de ocupación: relación porcentual entre libros prestados y stock general.
- Porcentaje de socios sancionados: tasa de morosidad activa sobre el padrón de socios vigentes.

DESCRIPCIÓN TRIGGERS

trg_actualizar_stock: AFTER INSERT / AFTER UPDATE en PRÉSTAMO:

- AFTER INSERT: si se agrega un nuevo registro a préstamo con estado 'activo' se va a decrementar 1 en el stock disponible del libro correspondiente al ejemplar prestado.
- AFTER UPDATE: si se actualiza un registro de préstamo y este pasa de 'activo' a 'devuelto' se va a incrementar 1 en el stock disponible del libro correspondiente.

trg_estado_socio: AFTER INSERT en SANCIÓN:

Cuando se agrega una nueva sanción para un socio en estado 'activo', el trigger cambia automáticamente su estado a 'SUSPENDIDO'. Esto refuerza la regla de negocio que indica que un socio con sanciones activas no puede realizar nuevos préstamos.

trg_audit_prestamo: AFTER INSERT / UPDATE / DELETE en PRÉSTAMO:

Registra en la tabla auditoria_prestamos cada operación realizada sobre préstamo, indicando el tipo de operación, el estado anterior, el estado nuevo, el timestamp y el usuario de base de datos que la ejecutó:

- AFTER INSERT: registra la creación, con estado_viejo = NULL.
- AFTER UPDATE: registra la actualización, con un estado viejo y nuevo.
- AFTER DELETE: registra la eliminación, con un estado_nuevo = NULL.

ARQUITECTURA DEL AGENTE IA

El agente ha sido diseñado utilizando el enfoque de Text-to-SQL. Su arquitectura se divide en las siguientes tres capas:

Capa de interfaz (Presentación): se implementa mediante Jupyter Notebooks y actúa como entorno de ejecución donde el usuario interactúa con el agente.

Capa de procesamiento (Inteligencia): utiliza el modelo gemini 2.5 flash de Google, el cual traduce las consultas del usuario a consultas SQL.

- **Celda 1 — Conexión a la base de datos**

Carga las variables de entorno desde el archivo .env usando python-dotenv y establece la conexión a MySQL en Aiven con SSL. La función get_connection() es reutilizada por todas las funciones del notebook.

- **Celda 2 — Inicialización del cliente Gemini y System Prompt**

Se instancia el cliente de la API de Google Gemini usando la clave almacenada en LLM_API_KEY. El SYSTEM_PROMPT incluye: descripción del rol, reglas de comportamiento, el esquema completo de la base de datos (tablas, columnas, constraints y vistas disponibles), y tres ejemplos de few-shot prompting (pregunta → SQL esperado).

La inclusión del esquema directamente en el prompt es la técnica central del agente: le permite al modelo razonar sobre la estructura de datos sin acceso directo a la base. Los ejemplos few-shot fueron elegidos para cubrir tres patrones distintos: uso de vista con LIMIT, uso de DISTINCT, y uso de LIKE para búsqueda parcial.

- **Celda 3 — Función text_to_sql()**

Recibe una pregunta en lenguaje natural, la concatena al SYSTEM_PROMPT y la envía al modelo gemini-2.5-flash. Limpia la respuesta de posibles bloques de código markdown (backticks) que el modelo puede agregar por defecto.

- **Celda 4 — Función ejecutar_consulta()**

Ejecuta una consulta SQL sobre MySQL usando pandas.read_sql() y devuelve el resultado como un DataFrame. Maneja excepciones retornando un DataFrame con la descripción del error, lo que permite que el notebook continúe aunque una consulta falle. La conexión se cierra siempre en el bloque finally.

- **Celda 5 — Función agente_responder()**

Función orquestadora que integra las dos anteriores: muestra la pregunta, el SQL generado (opcionalmente) y el resultado formateado. Es el punto de entrada principal para todas las pruebas del agente.

Capa de persistencia (Base de Datos): se encuentra hospedada en Aiven y separa la lógica transaccional de la lógica analítica usando stored procedures y vistas respectivamente. Estas últimas son exclusivamente utilizadas por el agente para garantizar la seguridad y la integridad de los datos.

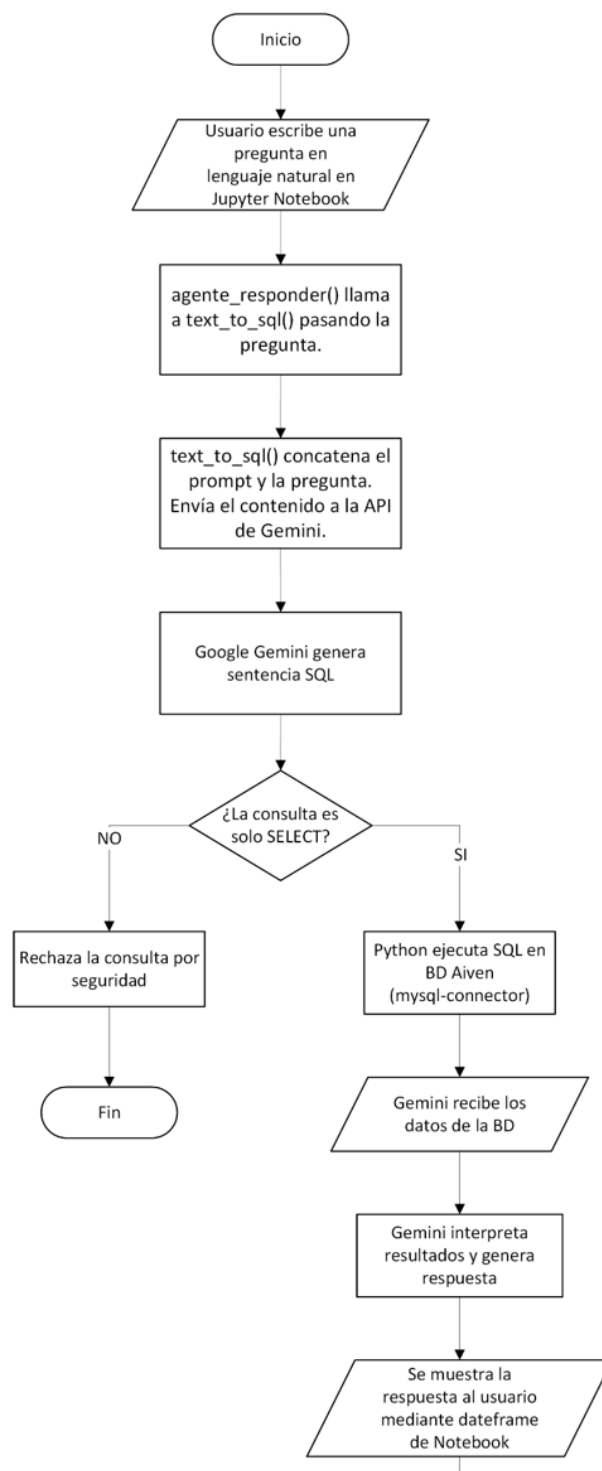
DECISIONES DE IMPLEMENTACIÓN

Variables de entorno: todas las credenciales (base de datos y API key) se almacenan en un archivo .env excluido del repositorio mediante .gitignore, siguiendo buenas prácticas de seguridad.

Modelo elegido: se utilizó gemini-2.5-flash por su disponibilidad en el tier gratuito de Google AI Studio y su buena capacidad de razonamiento para tareas Text-to-SQL.

Vistas como capa de abstracción: el esquema provisto al modelo incluye descripciones de las vistas disponibles. Esto reduce la complejidad de las consultas generadas y mejora la precisión, ya que el modelo puede responder muchas preguntas con un simple SELECT sobre una vista en lugar de construir JOINS complejos.

Manejo de errores: ejecutar_consulta() captura excepciones y las muestra como DataFrame, evitando que el notebook se interrumpa ante consultas inválidas.

Diagrama de flujo del proceso Text-to-SQL:

PROMPT DEL SISTEMA UTILIZADO

El prompt es el encargado de guiar el comportamiento del agente. Se indica cuál es la tarea a realizar y el esquema de la base de datos a utilizar. De esta manera, se le otorga al modelo el contexto necesario para ser preciso en sus respuestas.

```
SYSTEM_PROMPT = ""
Sos un experto en bases de datos MySQL. Tu única tarea es convertir
preguntas
en español a consultas SQL válidas para la base de datos 'biblioia'.

REGLAS:
- Respondé ÚNICAMENTE con la consulta SQL, sin explicaciones ni
comentarios.
- No uses bloques de código ni backticks.
- Preferí usar las VISTAS disponibles cuando corresponda.
- Si la pregunta no se puede responder con el esquema dado, respondé:
  SELECT 'No puedo responder esa pregunta con los datos disponibles';

=== ESQUEMA ===

GENERO (id_genero INT PK, nombre VARCHAR(60) UNIQUE NOT NULL,
descripcion VARCHAR(255))
AUTOR (id_autor INT PK, nombre VARCHAR(80) NOT NULL, apellido
VARCHAR(80) NOT NULL, nacionalidad VARCHAR(60))
LIBRO (isbn VARCHAR(20) PK, titulo VARCHAR(200) NOT NULL,
anio_publicacion YEAR, stock_total SMALLINT, stock_disponible SMALLINT)
  -- stock_disponible <= stock_total siempre
LIBRO_AUTOR (isbn FK->LIBRO, id_autor FK->AUTOR) -- N:M
LIBRO_GENERO (isbn FK->LIBRO, id_genero FK->GENERO) -- N:M
SOCIO (id_socio INT PK, dni VARCHAR(15) UNIQUE, nombre VARCHAR(80),
apellido VARCHAR(80), email VARCHAR(120) UNIQUE, fecha_alta DATE,
estado VARCHAR(12))
  -- estado: 'ACTIVO', 'SUSPENDIDO', 'BAJA'
EJEMPLAR (id_ejemplar INT PK, isbn FK->LIBRO, nro_ejemplar SMALLINT,
estado_fisico VARCHAR(12))
  -- estado_fisico: 'BUENO', 'DETERIORADO', 'BAJA'
```

```

PRESTAMO (id_prestamo INT PK, id_socio FK->SOCIO, id_ejemplar
FK->EJEMPLAR, fecha_prestamo DATE, fecha_vencimiento DATE,
fecha_devolucion DATE NULL, estado VARCHAR(12))
    -- estado: 'ACTIVO', 'DEVUELTO', 'VENCIDO'
SANCION (id_sancion INT PK, id_socio FK->SOCIO, tipo VARCHAR(20),
fecha_inicio DATE, fecha_fin DATE, motivo VARCHAR(255))
    -- tipo: 'MORA', 'DAÑO', 'PERDIDA', 'OTRO'
    -- activa cuando fecha_fin >= CURRENT_DATE
AUDITORIA_PRESTAMOS (id_audit INT PK, id_prestamo INT, operacion
VARCHAR(10), estado_nuevo VARCHAR(12), estado_viejo VARCHAR(12),
usuario_bd VARCHAR(80), fecha_hora DATETIME)

=== VISTAS DISPONIBLES ===

v_prestamos_vencidos (id_prestamo, dni, socio, titulo, fecha_prestamo,
fecha_vencimiento, dias_de_mora)
    -- préstamos con estado='VENCIDO' y sin devolución

v_prestamos_activos (id_prestamo, id_socio, dni, socio, email, isbn,
titulo, nro_ejemplar, fecha_prestamo, fecha_vencimiento, dias_vencido)
    -- todos los préstamos con estado='ACTIVO'

v_libros_disponibles (isbn, titulo, stock_disponible, generos, autores)
    -- libros con stock_disponible > 0 y al menos un ejemplar en buen
estado

v_historial_socios (id_socio, dni, socio, isbn, titulo, fecha_prestamo,
fecha_vencimiento, fecha_devolucion, estado_prestamo)
    -- historial completo de préstamos por socio

v_libros_mas_prestados (isbn, titulo, autores, total_prestamos)
    -- ranking de libros por cantidad de préstamos

v_socios_sancionados (id_socio, dni, socio, estado, tipo, fecha_inicio,
fecha_fin, motivo, dias_restantes)
    -- socios con sanciones activas hoy

v_autores_prolifricos (id_autor, autor, nacionalidad, cantidad_libros)
    -- autores con más de 1 libro en la biblioteca

v_lista_socios (lista_socios)
    -- contine todas las filas de la tabla SOCIO

```

```
v_reporte_salud_biblioteca (total_inventario, ejemplares_circulacion,
tasa_ocupacion, total_socios, porcentaje_socios_sancionados)
-- contiene una única fila con las métricas generales y el estado de
salud en tiempo real de la biblioteca

=== EJEMPLOS ===

Pregunta: ¿Cuáles son los 5 libros más prestados este año?
SQL: SELECT isbn, titulo, total_prestamos FROM v_libros_mas_prestados
LIMIT 5;

Pregunta: ¿Qué socios tienen préstamos vencidos en este momento?
SQL: SELECT DISTINCT dni, socio FROM v_prestamos_vencidos;

Pregunta: ¿Qué libros de ciencia ficción están disponibles para
prestar?
SQL: SELECT isbn, titulo, stock_disponible FROM v_libros_disponibles
WHERE generos LIKE '%Ciencia Ficción%';
"""
```

EVALUACIÓN DEL AGENTE

Se evaluaron 10 preguntas formuladas en lenguaje natural, cubriendo los casos mínimos exigidos por el enunciado más preguntas de elaboración propia. Para cada una se registra la consulta SQL generada por Gemini 2.5 Flash, el resultado obtenido sobre la base de datos y un análisis de correctitud.

Cada pregunta fue enviada al agente mediante la función `agente_responder()`, que internamente llama a `text_to_sql()` para obtener el SQL y luego a `ejecutar_consulta()` para ejecutarlo sobre MySQL. Se consideró una respuesta correcta cuando:

- El SQL generado es sintácticamente válido y ejecutable.
- El resultado coincide con la respuesta esperada al consultar la base manualmente.
- Se prioriza el uso de vistas disponibles cuando corresponde.

Tabla de resultados

#	Pregunta en lenguaje natural	SQL generado por Gemini	Resultado	Análisis
1	¿Cuáles son los 5 libros más prestados este año?	SELECT isbn, titulo, total_prestamos FROM v_libros_mas_prestados LIMIT 5;	5 filas: Sapiens (4), Clean Code (3), El Aleph (3), Ficciones (3), ¿Sueñan los androides...? (3)	Correcto. Usó la vista adecuada y aplicó LIMIT correctamente.
2	¿Qué socios tienen préstamos vencidos en este momento?	SELECT DISTINCT dni, socio FROM v_prestamos_vencidos;	10 filas: Benjamín Gutiérrez, Renata Mendoza, Axel Silva, Brenda Cabrera, Rodrigo Aguilar, Zoe Vega, Sebastián Reyes, Lucas Fernández, Valentina Gómez, Matías Rodríguez	Correcto. Uso de DISTINCT para evitar duplicados. Identificó correctamente la vista apropiada.
3	¿Cuántos ejemplares disponibles hay del libro con ISBN 978-0-13-235088-4?	SELECT stock_disponible FROM v_libros_disponibles WHERE isbn = '978-0-13-235088-4';	1 fila: stock_disponible = 3	Correcto. Filtró correctamente por ISBN en la vista de disponibilidad.
4	¿Qué libros de ciencia ficción están disponibles para prestar?	SELECT isbn, titulo, stock_disponible FROM v_libros_disponibles WHERE generos LIKE '%Ciencia Ficción%';	7 filas: La mano izquierda..., Los desposeídos, 2001: Una odisea..., Cita con Rama, Fundación, El fin de la eternidad, ¿Sueñan los androides...?	Correcto. Uso adecuado de LIKE con el campo generos de la vista.
5	¿Cuál es el historial de préstamos del socio con DNI 30100001?	SELECT isbn, titulo, fecha_prestamo, fecha_vencimiento, fecha_devolucion, estado_prestamo FROM v_historial_socios WHERE dni = '30100001'	3 filas: Fundación (ACTIVO), El fin de la eternidad (VENCIDO), Fundación (DEVUELTO)	Correcto. Historial completo con todos los estados del socio.

#	Pregunta en lenguaje natural	SQL generado por Gemini	Resultado	Análisis
6	¿Qué autores tienen más de 3 libros en la biblioteca?	SELECT autor, cantidad_libros FROM v_autores_prolíficos WHERE cantidad_libros > 3;	1 fila: Jorge Luis Borges (4)	Correcto. Filtro adicional sobre la vista que ya filtra > 1 libro.
7	¿Cuántas sanciones se generaron en el último mes?	SELECT COUNT(*) FROM SANCION WHERE fecha_inicio >= DATE_SUB(CURDATE(), INTERVAL 1 MONTH)	1 fila: COUNT(*) = 1	Correcto. Consultó directamente la tabla SANCION con filtro de fecha dinámica.
8	¿Cuál es el género preferido por el socio con DNI 30100002?	SELECT G.nombre AS genero_preferido FROM v_historial_socios VHS JOIN LIBRO_GENERO LG ON VHS.isbn = LG.isbn JOIN GENERO G ON LG.id_genero = G.id_genero WHERE VHS.dni = '30100002' GROUP BY G.nombre ORDER BY COUNT(*) DESC LIMIT 1;	1 fila: genero_preferido = Ciencia Ficción	Correcto. El modelo combinó la vista de historial con tablas auxiliares para inferir preferencia por frecuencia.
9	¿Cuál es el porcentaje de socios sancionados?	SELECT porcentaje_socios_sancionados FROM v_reporte_salud_biblioteca;	1 fila: porcentaje_socios_sancionados = 0.0	Correcto. Identificó la vista de reporte. El valor 0.0 es válido ya que no hay sanciones activas al momento de la consulta.
10	¿Cuál es la tasa de ocupación de ejemplares?	SELECT tasa_ocupacion FROM v_reporte_salud_biblioteca;	1 fila: tasa_ocupacion = 48.39	Correcto. Extrajo correctamente la columna específica del reporte de salud.

El agente respondió correctamente las 10 preguntas evaluadas, alcanzando una tasa de acierto del 100%. Se destacan los siguientes puntos:

- Uso consistente de vistas: en 8 de las 10 preguntas, Gemini identificó correctamente cuál vista utilizar en lugar de construir JOINS desde cero.

- Consultas sobre tablas base: cuando la pregunta no tenía vista directa el modelo construyó la consulta correctamente combinando tablas y vistas auxiliares.

MÓDULO DE RECOMENDACIONES

El módulo de recomendaciones tiene tres funciones que trabajan en cadena:

obtener_perfil_socio(id_socio) consulta las vistas `v_generos_por_socio` y `v_autores_por_socio` para saber qué géneros y autores leyó el socio a lo largo de toda su historia de préstamos, sin importar si los devolvió o no. Devuelve un diccionario con dos listas.

obtener_libros_candidatos(id_socio) consulta `v_libros_no_leidos_por_socio`, que filtra todos los libros con stock disponible y excluye los que el socio ya prestó alguna vez usando un `NOT EXISTS`. Así garantizas que no le recomendás algo que ya leyó.

recomendar_para(id_socio) obtiene el perfil y los candidatos, verifica que el socio tenga historial (si no, avisa y corta), construye un prompt donde le dice a Gemini que actúe como bibliotecario con el contexto del perfil y la lista de candidatos, y le pide las 3 mejores recomendaciones con justificación. Gemini devuelve texto libre que se imprime directamente.

Utilizamos las tres vistas definidas previamente en vez de escribir las consultas en el código Python. Las vistas encapsulan toda la lógica de negocio (qué significa "leído", qué significa "disponible", qué significa "no leído por este socio") y el Python solo consume los resultados.

También decidimos pasarle a Gemini el perfil completo en lenguaje natural en vez de solo los títulos candidatos. Eso le permite razonar sobre afinidades de estilo y temática entre lo que ya leyó el socio y lo que le estás ofreciendo, que es exactamente lo que hace un buen bibliotecario humano.

Ejemplo de ejecución

Se ejecutó el módulo para el socio con ID

Perfil detectado:

- Géneros favoritos: Ciencia Ficción
- Autores leídos: Jorge Luis Borges, Isaac Asimov

Recomendaciones generadas por Gemini:

1. **2001: Una odisea del espacio** — Este es un clásico absoluto que te transportará al espacio profundo y te hará reflexionar sobre la evolución humana y el universo. Con la

maestría de Arthur C. Clarke, disfrutarás de una narrativa épica que, al igual que Asimov, combina ciencia rigurosa con misterios cósmicos y un sentido de la maravilla que te dejará sin aliento.

2. ¿Sueñan los androides con ovejas eléctricas? — Esta novela te sumergirá en un futuro distópico y te hará cuestionar qué significa ser humano en un mundo post-apocalíptico. Su exploración de la identidad, la empatía y la realidad artificial te recordará la profundidad reflexiva y a veces laberíntica de Borges, pero con un toque futurista y psicológico fascinante.

3. Los desposeídos — Una obra maestra que te invita a explorar sociedades utópicas y distópicas, y a reflexionar sobre la política, la libertad y la filosofía a través de la ciencia ficción. Ursula K. Le Guin construye mundos complejos que son metáforas para nuestra propia existencia, con la inmersión y construcción de mundos que un fan de la ciencia ficción apreciará profundamente.

Módulo de Recomendaciones Colaborativas

Como extensión del módulo anterior, se implementó un segundo enfoque de recomendación basado en filtrado colaborativo. A diferencia del módulo por historial individual, este identifica socios con patrones de lectura similares al socio consultado y sugiere libros que esos lectores afines leyeron pero el socio actual todavía no.

La función `obtener_libros_comunidad(id_socio)` realiza una consulta que en tres pasos: primero identifica los géneros que leyó el socio actual, luego encuentra otros socios que leyeron esos mismos géneros, y finalmente obtiene los libros que esos socios afines prestaron pero que el socio actual no tiene en su historial y que cuentan con stock disponible. El resultado se limita a 6 candidatos para no sobrecargar el prompt.

La función `recomendar_colaborativo(id_socio)` toma esos candidatos, construye un prompt donde Gemini actúa como bibliotecario y menciona explícitamente que las recomendaciones provienen de lectores con gustos similares, y genera 3 sugerencias con justificación.

CONCLUSIONES

El desarrollo del presente trabajo práctico ha permitido integrar y aplicar los conceptos fundamentales de base de datos en sinergia con inteligencia artificial generativa, tales como el diseño relacional con restricciones de integridad, triggers, stored procedures, transacciones y propiedades ACID. Asimismo,

se ha implementado la abstracción de datos mediante vistas lo cual no solo sirve para ocultar complejidad al usuario, sino que constituye una herramienta de seguridad para limitar el alcance del modelo de inteligencia artificial, garantizando que el agente opere bajo el principio del menor privilegio al limitarse a operaciones de sólo lectura.

En relación a los aprendizajes adquiridos, se han implementado triggers y stored procedures, necesarios para la integridad de los datos y encapsular la lógica del negocio, respectivamente. Paralelamente, se consolidaron competencias en el manejo de entornos de desarrollo (Visual Studio Code), control de versiones (GitHub) e integración con API y base de datos en la nube, herramientas que constituyen pilares esenciales para afrontar desafíos en el mundo laboral actual.

Por otro lado, la mayor limitación detectada fue la cuota de uso del servidor gratuito de la API luego de una cierta cantidad de consultas. Por lo cual, era necesario esperar hasta que se vuelva a restablecer dicha cuota.

Para finalizar, se puede implementar una base de datos intermedia de manera tal que si el usuario realiza la misma consulta en menos de una hora, el sistema devuelve la respuesta guardada en caché en lugar de preguntar al agente nuevamente. De esta manera, se ahorran consultas y tiempo de espera.

BIBLIOGRAFÍA

Fazzio, F., & Colombo, P. (2026). *Integridad de datos* [Diapositivas]. UTN FRCU.

Fazzio, F., & Colombo, P. (2026). *Stored Procedures y Triggers* [Diapositivas]. UTN FRCU.

Fazzio, F., & Colombo, P. (2026). *Transacciones* [Diapositivas]. UTN FRCU.

Fazzio, F., & Colombo, P. (2026). *Vistas en SQL* [Diapositivas]. UTN FRCU.